Отчёт по индивидуальному заданию №3

Моделирование работы супермаркета с использованием взаимодействующих процессов

Исполнитель: Батышкина Алёна Игоревна **Группа:** БПИ253 **Вариант:** 8 **Дата:** 09.11.2026

---

1. Условие задачи

Разработать консольное приложение на языке C, моделирующее процесс обслуживания покупателей в супермаркете. Программа должна состоять из взаимодействующих процессов, использовать системные вызовы, разделяемую память (POSIX Shared Memory) и синхропримитивы (POSIX Semaphores) для согласованной работы. Также требуется предусмотреть обработку прерываний (SIGINT) и корректное освобождение ресурсов. Ключевые сущности: Покупатели, Кассиры, Очереди ограниченной вместимости.

---

2. Сценарий моделирования и отображение в процессы

В отличие от предыдущих заданий (где использовались потоки), здесь каждая сущность реализована как отдельный **процесс**. Так как процессы имеют изолированные адресные пространства, обмен данными ведется через разделяемую память, а синхронизация — через семафоры.

Сущность	Представление в коде	Поведение (процесс)
Генератор	`role = "generator"`	Создает процессы покупателей с заданным интервалом, контролирует закрытие магазина (`entrance_closed`).
Покупатель	`role = "customer"`	Имитирует выбор товаров (`usleep`), выбирает очередь, ждет обслуживания, завершается после получения сигнала.
Кассир	`role = "cashier"`	Ждет покупателя, извлекает его из очереди, имитирует обслуживание, обновляет статистику, будит покупателя.
Дисплей	`role = "display"`	Фоновый процесс. Каждые 50 мс считывает состояние из разделяемой памяти и перерисовывает таблицу.

Инварианты:

1. Один кассир обслуживает не более одного покупателя.
2. Покупатель находится не более чем в одной очереди.
3. Вместимость очередей не превышается.

4. Обслуженный покупатель покидает систему навсегда.

---

3. Структуры данных и разделяемая память

Главная структура `Simulation` находится в разделяемой памяти. **Важное архитектурное решение:** в структурах, лежащих в разделяемой памяти, **запрещены указатели**, так как адресные пространства процессов различаются. Поэтому очереди реализованы через статические массивы (`int customer_ids[MAX_QUEUE_CAPACITY]`).

Для выделения памяти используются системные вызовы:

- `shm_open()` — создание объекта разделяемой памяти.
- `ftruncate()` — выделение нужного размера под структуру `Simulation`.
- `mmap()` — отображение памяти в адресное пространство процесса.

Структура Simulation содержит в себе массивы структур Customer (покупатели), Cashier (кассиры), Queue (очереди с ограниченной вместимостью), а также объект Statistics для сбора метрик. Все эти данные синхронизируются семафорами, которые также описаны в структуре.

4. Реализация синхронизации

Для защиты общих данных и сигнализации используются POSIX семафоры.

1. **Мьютексы (защита данных):**
   - `mutex_time` — защищает счетчик модельного времени.
   - `mutex_stats` — защищает глобальную статистику.
   - `mutex_queues[i]` — защищает доступ к конкретной очереди (добавление/извлечение).
2. **Сигнальные семафоры (события):**
   - `customers_ready[i]` — кассир засыпает на этом семафоре, покупатель делает `post`, когда встает в очередь.
   - `service_done[id]` — покупатель засыпает здесь, кассир делает `post`, когда заканчивает обслуживание.

---

5. Реализация трех уровней сложности (согласно методичке)

5.1. Базовая версия на 4-5 баллов

Реализована в бинарнике `./supermarket`.

- **Схема запуска:** Родительский процесс делает `fork()` для каждого кассира и покупателя.
- **Семафоры: Неименованные** POSIX семафоры (`sem_init` с флагом `pshared=1`). Они физически лежат внутри структуры `Simulation` в разделяемой памяти. Это позволяет легко обмениваться ими между процессами `fork`.

5.2. Версия на 6 баллов (Именованные семафоры)

Реализована в бинарнике `./supermarket_named` (собирается с флагом `-DUSE_NAMED_SEM`).

- **Схема запуска:** Также использует `fork()`.
- **Семафоры: Именованные** POSIX семафоры (`sem_open`, `sem_close`). Они хранятся в ядре ОС (в `/dev/shm/`).
- *Особенность:* Для каждого процесса была написана функция `sim_attach_named_sems()`, которая заново открывает семафоры по их именам, чтобы избежать передачи невалидных указателей.

5.3. Версия на 7-8 баллов (Независимые процессы)

Для соответствия требованиям создан bash-скрипт **`run.sh`**.

- **Схема запуска:** Процессы **не являются дочерними** друг для друга. Скрипт `run.sh` запускает их независимо в фоновом режиме:
  1. `./supermarket_named init` — инициализирует память.
  2. `./supermarket_named display &`
  3. `./supermarket_named cashier 0 &`
  4. `./supermarket_named cashier 1 &`
  5. `./supermarket_named generator &`
- **Синхронизация:** Используются **именованные** семафоры.
- **Завершение:** Скрипт перехватывает `Ctrl+C` (`trap`), убивает фоновые процессы и вызывает `./supermarket_named cleanup`, который собирает статистику и делает `shm_unlink`.

---

6. Пользовательский интерфейс и логирование

6.1. Конфигурационный файл

Параметры задаются в файле (аналогично предыдущим ИДЗ): `total_customers`, `queue_capacity`, диапазоны времени, количество товаров, стратегия выбора очереди (SHORTEST, FIRST,

RANDOM) и т.д.

6.2. Табличный интерфейс

Процесс `display` отрисовывает таблицу, обновляя её на одном месте с помощью ANSI escape-последовательностей (`\033[H`):

- Заголовок с текущим временем.
- Общая статистика (Прибыло / Обслужено / Ушло).
- Состояние касс (список ID в очереди, статус `serving` или `sleeping`).

6.3. Логирование

Все действия записываются в файл `simulation.log` с использованием системного вызова `write()`, что гарантирует атомарность записи из разных процессов.

---

7. Варианты запуска и завершения

Действие	Команда	Поведение
Независимые процессы	`make run`	Запускает `run.sh` (именованные семафоры, 7-8 баллов).
Тестирование	`make test`	Прогоняет 18 тестов через `run.sh`, проверяя закон сохранения.
Прерывание	`Ctrl+C`	Перехватывается `SIGINT`. В `run.sh` срабатывает `trap`, убивающий фоновые процессы. Семафоры и память очищаются.
Очистка	`./supermarket_named cleanup`	Выводит статистику, удаляет сегменты памяти (`shm_unlink`) и семафоры.

---

8. Результаты работы программы

Пример промежуточного состояния интерфейса во время симуляции

```
|============================================================|
  | SUPERMARKET SIMULATION                                     |
  | Time: 0245 / 1800                                          |
  |============================================================|
  | Arrived: 28   Served: 12   Left: 16                        |
  |============================================================|
  | Cashier 0: [C2][C5][C8]              serving C2            |
  | Cashier 1: [C3][C7]                  sleeping              |
  |============================================================|
  | Recent events: (see simulation.log)                        |
  |                                                            |
  |                                                            |
  |                                                            |
  |============================================================|
```

Пример итоговой статистики:

```
*****************************
  SIMULATION STATISTICS
  *****************************
  Simulation duration:   169 sec
  Total arrived:         50
  Total served:          34
  Total left (no serv.): 16
  Average shopping time: 19.26 sec
  Max queue 0 size:      5
  Max queue 1 size:      5
  Cashier 0 served:      16
  Cashier 1 served:      18
  *****************************

  Conservation law verified: 50 = 34 + 16
```

9. Критическая оценка кода и использование ИИ

В ходе финального ревью и тестирования были выявлены и устранены следующие критические недочёты и архитектурные проблемы:

1. **Segmentation fault при переходе к независимым процессам (Именованные семафоры).**

Проблема: Изначально указатели на семафоры (`sem_t *`), полученные через `sem_open`, сохранялись в разделяемую память. При независимом запуске (через bash-скрипт) процессы читали эти указатели, но они были валидны только внутри процесса-создателя. Это приводило к падению программы. *Решение:* Был изменен подход. Теперь каждый процесс (кассир, диспетчер) при старте заново открывает семафоры по их строкам-именам с помощью функции `sim_attach_named_sems()`.

2. **Вечная блокировка (Deadlock) при завершении работы.**

   Проблема: Когда генератор заканчивал создавать покупателей, кассиры, дообслужив последних, засыпали на `sem_wait()` и не знали, что магазин уже закрывается. Родительский процесс зависал в ожидании их завершения. *Решение:* Был добавлен флаг `entrance_closed`. После завершения генератора всем кассирам принудительно отправляется сигнал `sem_post()`, чтобы они проснулись, проверили флаг и корректно завершили работу.

3. **Утечка ресурсов и некорректное завершение по Ctrl+C.**

   Проблема: При нажатии `Ctrl+C` сигнал убивал только основной bash-скрипт `run.sh`, а фоновые процессы (кассиры, дисплей) оставались висеть в памяти, как и POSIX-семафоры. *Решение:* В скрипт `run.sh` добавлен обработчик сигналов (`trap`), который перехватывает `SIGINT`, принудительно завершает все фоновые процессы и вызывает программу очистки (`cleanup`), удаляя семафоры и разделяемую память.

4. **Отсутствие инициализации ГПСЧ в процессе-генераторе.**

   Проблема: Процесс `generator` не вызывал `rng_init()`, из-за чего при каждом запуске использовались одинаковые интервалы времени (дефолтный seed). *Решение:* Добавлен вызов `rng_init(sim->config.random_seed)` в начале работы процесса-генератора.